

A Gene Index That Moved

How the 010 design-space predictions became invalid, what the construction panel is actually predicted to do, and what the model can still be used for

Michael Volk

September 3, 2026

Abstract

The predictions that selected experiment 010's construction panel are invalid, for two independent reasons that compound. The genome gene set shrank from 6,607 entries to 6,579 between inference runs, and the 28 absent entries are mitochondrial open reading frames that sort first, so every gene index shifted by exactly 28 and each triple was scored as a different triple. Separately, the candidate genes were drawn from a list that was never intersected with the model's gene space, and a gene the index cannot resolve contributes no perturbation at all, so those records were silently scored as doubles. Of 432 stored panel combinations, 266 were scored as doubles and 6 as singles. Both defects are confirmed rather than inferred: re-scoring stored triples through a validated path reproduces each run under exactly one index map and not the other, at correlation 0.999998.

Rescoring the panel correctly changes what should be built. Across the 50 combinations shared with the stored files the corrected values correlate with the recorded ones at 0.115, and on the 20-triple build list the recorded values span 0.409 to 0.711 while the corrected ones span -0.134 to $+0.011$ against a training-label standard deviation of 0.063. Nineteen of the 20 are predicted negative and the single positive is not agreed across checkpoints, so the list contains no triple the model confidently calls positive.

What the model can still do is retrieve negative interactions, scored against the published call, $\tau < -0.08$ with $p < 0.05$. Among the 100 records each checkpoint calls most negative, 32 to 38 percent are real calls against a base rate of 1.7 percent, and the transformer's margin over an additive null is proportionally larger there than on correlation. That holds only on the split whose leakage the companion document quantifies. Once query doubles are made disjoint the baselines fall to 1.5 to 1.8 times the base rate, which is close to chance, and the transformer has not been refit there.

Section status:  ready  author-reviewed, keep stable  not done, agents may edit freely

Contents

1	What broke 	2
1.1	The index space is rebuilt, not stored 	2
1.2	Defect one, a uniform shift of 28 	2
1.3	Defect two, triples scored as doubles 	2
1.4	A name problem, not a missing gene 	2
2	The evidence 	3
2.1	The reference the test is anchored on 	3
2.2	The decisive test 	3
2.3	What is invalid and what is not 	3
2.4	The guard, and the fix it is not 	3
3	What the panel is actually predicted to do 	4
3.1	The plate carries one locus twice 	4
3.2	The corrected ranking, and how far it can be read 	4
3.3	The 20 triples on the build list 	4
4	What the model can still be used for 	7
4.1	The measurement that is evidence 	7
4.2	It does not survive the disjoint split 	7
4.3	What this supports 	8
5	Established, and not 	8
5.1	Established 	8
5.2	Not established 	8
5.3	Next 	8

1 What broke ✗

Experiment 010 trained an equivariant cell graph transformer on 376,732 Kuzmin trigenic records and then scored three unmeasured design spaces with it. The largest, `inference_3`, covered 465,735,532 triples genome-wide and is what the panel-12 and panel-24 gene selections were drawn from. Those selections went to the bench.

Every prediction in that run is wrong, in two ways that compound.

1.1 The index space is rebuilt, not stored ✗

The model addresses genes by position. Its embedding table has 6,607 rows, read directly from the checkpoint, and a record's perturbed genes arrive as integer positions in the cell graph's sorted node list. Nothing ties the two together. The cell graph is constructed at run time from whatever gene set the genome object currently yields, and no assertion compares its node count against the `gene_num` the checkpoint was built for.

So the index space is a derived quantity recomputed at every use, not an artifact stored alongside the build. When the genome moves, every embedding row silently comes to mean a different gene.

1.2 Defect one, a uniform shift of 28 ✗

Between January and February 2026 the genome gene set went from 6,607 entries to 6,579. The two `inference_2` and `inference_3` dataset builds logged the smaller number at build time. The 28 missing entries are the mitochondrial open reading frames Q0010 through Q0297, and because they sort before every nuclear systematic name they occupy positions 0 through 27.

The consequence is not partial. Of the 6,579 genes present in both sets, not one keeps its index and every one shifts by exactly 28. YAL001C sits at row 28 in the space the checkpoint learned and was fed as row 0. Twenty-eight embedding rows were never addressed at all.

1.3 Defect two, triples scored as doubles ✗

The triple generator drew its candidate genes from the Costanzo single-mutant gene list and never intersected them with the model's gene space. Those are different universes: of the 5,493 genes Costanzo measured, 5,440 are inside the model's 6,607 and 53 are outside. By our own annotation 35 of the 53 are absent from the GFF entirely, 11 are typed `pseudogene`, 6 `blocked_reading_frame` and 1 `transposable_element_gene`.

A gene the index cannot resolve does not raise. The perturbation processor builds its index list with a set comprehension over node names, so an unmatched name contributes nothing and the list is simply shorter, and the readout pools by mean over however many indices arrive. A two-element list is a perfectly valid double.

Among the 432 distinct panel combinations that were stored, 160 had all three genes indexable, 266 had two and 6 had one. They carry only 220 distinct prediction values. The collapse is visible without running anything:

```
YIL174W + YKL033W-A + YPL081W -> 0.711426
YJL017W + YKL033W-A + YPL081W -> 0.711426
YKL033W-A + YLR312C-B + YPL081W -> 0.711426
```

Three different triples, one value, because all three of YIL174W, YJL017W and YLR312C-B were unresolvable and each triple collapsed onto the double {YKL033W-A, YPL081W}.

1.4 A name problem, not a missing gene ✗

The distinction matters and is easy to state wrongly. Three of the five unresolvable panel names are simply outdated systematic names: YJL017W is now YJL016W, YKL200C is now YKL201C, and YLR312C-B is now YLR313C. R64 records YLR313C as one verified gene, SGD:S000004305, carrying Alias: ['SPH1', 'YLR312C-B'].

That gene has a trained embedding row, number 4389, and always did. What the original run lacked was a lookup entry for the outdated string, because the gene set it indexed against stores current systematic names. Rescoring under the resolved name is therefore not inventing a prediction for a gene the model never saw; it is asking the model about the locus the strain actually deletes.

The remaining two, YIL174W and YLL017W, are typed **pseudogene** in R64 and fall outside our gene set because that set is built from features of type **gene**. They are still real loci with SGD records, YLL017W being SDC25, and Costanzo built and measured deletion strains for both. Their absence is a property of how the gene set is constructed, not of the genes.

The same alias miss, a second time. The build list’s own training-support audit reports YLR312C-B as having zero trigenic training records and labels it extrapolation. That count was taken under the outdated name. The perturbed-gene index is keyed on current systematic names, so the lookup returns nothing; under YLR313C the gene has 9 records. The gene on that panel with genuinely zero trigenic training records is YER079W.

2 The evidence ✗

Both defects are confirmed rather than inferred. The confirmation matters because the first symptom, one checkpoint appearing to disagree with itself across runs, has an obvious wrong reading: that the model is nondeterministic. It is not.

2.1 The reference the test is anchored on ✗

A hand-built forward pass that computes the encoder once and feeds it only perturbed-gene indices reproduces the recorded validation metrics of all three checkpoints to the fourth decimal, for example 0.461917 against a recorded 0.461881. That agreement is what licenses the path as a reference: a wrong gene ordering or a mis-loaded weight would give a plausible but wrong number instead.

2.2 The decisive test ✗

Two independent 384-triple samples were taken from each stored prediction file, at different offsets, and re-scored through that reference under each candidate index map.

Run	sample	6,607-gene map	6,579-gene map
inference_1	head	0.999998	0.115
inference_1	middle	0.999999	−0.252
inference_3	head	0.074	0.999999
inference_3	middle	−0.003	0.999998

Table 1: Pearson correlation between each run’s stored predictions and the validated scoring path, under each candidate index space. Every run is reproducible under exactly one map and neither is reproducible under both. Mean absolute difference at the reproducing map is about 2×10^{-5} , which is the runs’ half-precision autocast.

Three things follow. The model is deterministic, since every stored prediction is exactly recoverable. **inference_1** is correct and **inference_2** and **inference_3** are not. And the perfect agreement between runs 2 and 3 that first looked reassuring was evidence they shared the same wrong map, not evidence either was right.

2.3 What is invalid and what is not ✗

Invalid: everything derived from the **inference_2** and **inference_3** prediction files, which is the panel-12 and panel-24 gene selections and every table and figure built from them.

Unaffected: **inference_1**, whose 526-gene panel lies entirely inside the model’s gene set, so it escaped both defects rather than only the first. Also unaffected is every number computed on the labeled split through the validated path, including the held-out metrics and the additive-null comparison.

2.4 The guard, and the fix it is not ✗

The model now raises when the cell graph’s gene node count differs from the **gene_num** the checkpoint was built for. That converts a silent relabeling of the entire genome into a failure at the first forward pass.

It is detection, not prevention. The condition it catches is a symptom of the design in Section 1.1, where the index space is recomputed from a live genome at every use. Pinning the ordering with the build and loading it would make

the failure impossible instead of merely loud. A commit hash cannot substitute: the library was untouched between the two runs, and what moved was the genome underneath them, so two runs at the same commit scored different genes.

3 What the panel is actually predicted to do ✗

Construction had already started when the defect was found, so the operative question is not which genes to pick but which combinations among the picked genes the model favors. Every combination was therefore rescored under the correct index space with all three checkpoints, rather than one, because two training runs share only 0.39 to 0.47 of their top 100.

3.1 The plate carries one locus twice ✗

The panel at the bench is not the model-selected panel-12. Read from the Echo transfer report of the plating run itself, it is twelve strain labels: COS111, ELC1, LCL1, MMS2, SPH1, YEH1, YER079W, YJR060W, YKL033W-A, YLR312C-B, YOS9 and YPL081W.

SPH1 and YLR312C-B are separate wells and, by Section 1.4, one gene. Twelve labels are eleven distinct loci. Either the same deletion is present twice under two names or one well is not the strain its label says, and only the bench can settle which.

3.2 The corrected ranking, and how far it can be read ✗

All 165 triples over the eleven loci fall between -0.19 and $+0.03$, against a training-label standard deviation of 0.063. Nothing on this plate is predicted far from the mean.

The three checkpoints agree on the sign of the interaction for 133 of the 165. That agreement is not evenly distributed. Among the top of the ranking most triples straddle zero and several have one checkpoint positive and another negative on the same triple; only YER079W+MMS2+YEH1 has all three clearly on one side, at $+0.032$ with a spread of 0.0035. The bottom is resolved: every one of the twelve most negative is negative in all three checkpoints, the strongest being MMS2+YJR060W+YEH1 near -0.19 .

So the model has a reproducible opinion about which triples are strongly negative and almost none about which are positive.

3.3 The 20 triples on the build list ✗

The instruction the bench is working from is 25 doubles plus 20 triples. Those 20 were ranked by `inference_3`, so the numbers that ordered them are not predictions of those triples.

	as listed	corrected
range over the 20	0.4092 to 0.7114	-0.1336 to $+0.0108$
predicted positive	20 of 20	1 of 20
all three checkpoints agree on sign	not available	17 of 20

Table 2: The build list before and after the index correction. The training-label standard deviation is 0.0633, so the listed values sat six to eleven of those above zero, where a shrunk squared-error fit does not put predictions.

The single positive is not usable. YGL087C+YLL012W+YPL046C has a corrected mean of $+0.0108$ with a spread of 0.0607 across the three checkpoints, and they do not agree on its sign. Every one of the 17 triples the checkpoints do agree on is negative, the strongest being YJR060W+YLL012W+YPL046C at -0.1336 , which the original list ranked twenty-first and therefore excluded.

Why the top six were the top six. Ranks 1 through 6 all contain YLR312C-B and all scored between 0.683 and 0.711. Each was scored as the double formed by its other two genes, by Section 1.3. Under the correct index the six land between -0.007 and -0.052 . Their rank was an artifact of the collapse, not a claim about them.

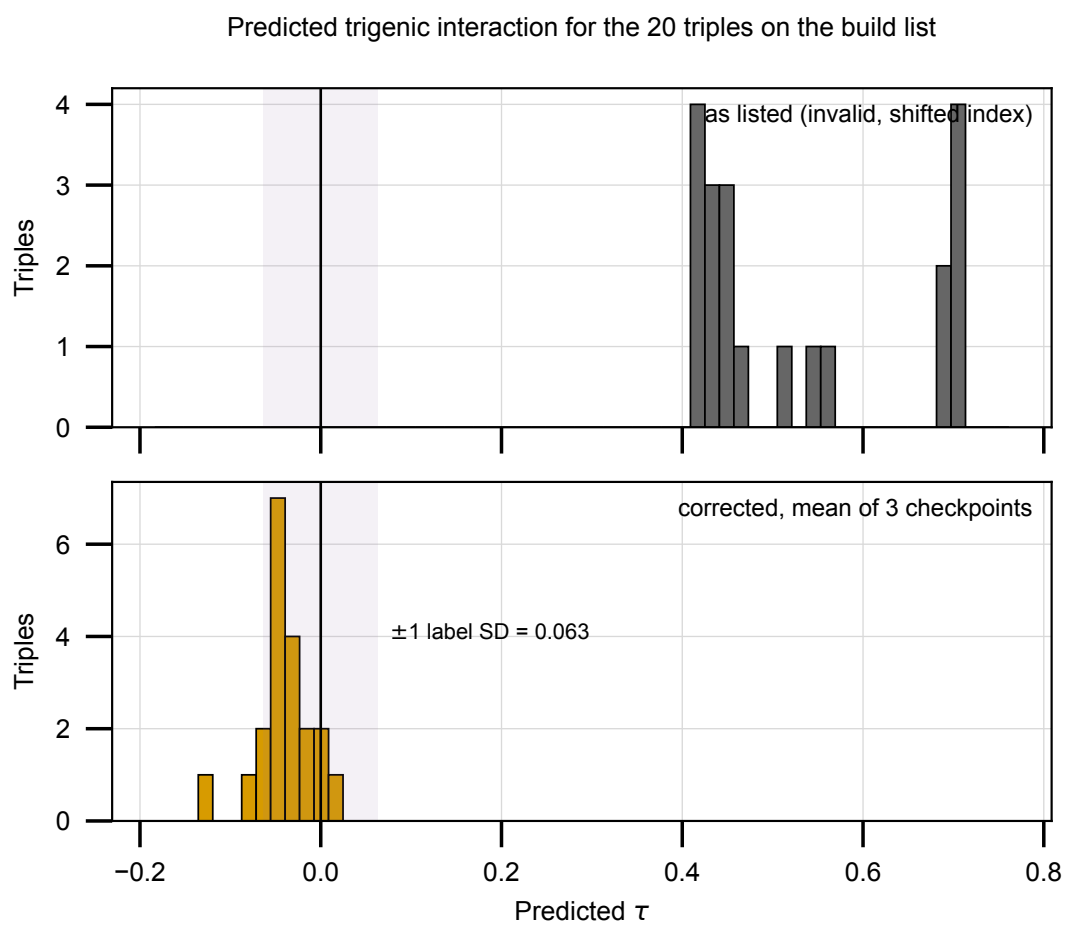


Figure 1: Predicted trigenic interaction for the 20 triples on the build list, both scorings on one axis. The shaded band is one training-label standard deviation.

The 20 build-list triples, re-ranked
red = a gene with no trigenic training labels (2 of 20)

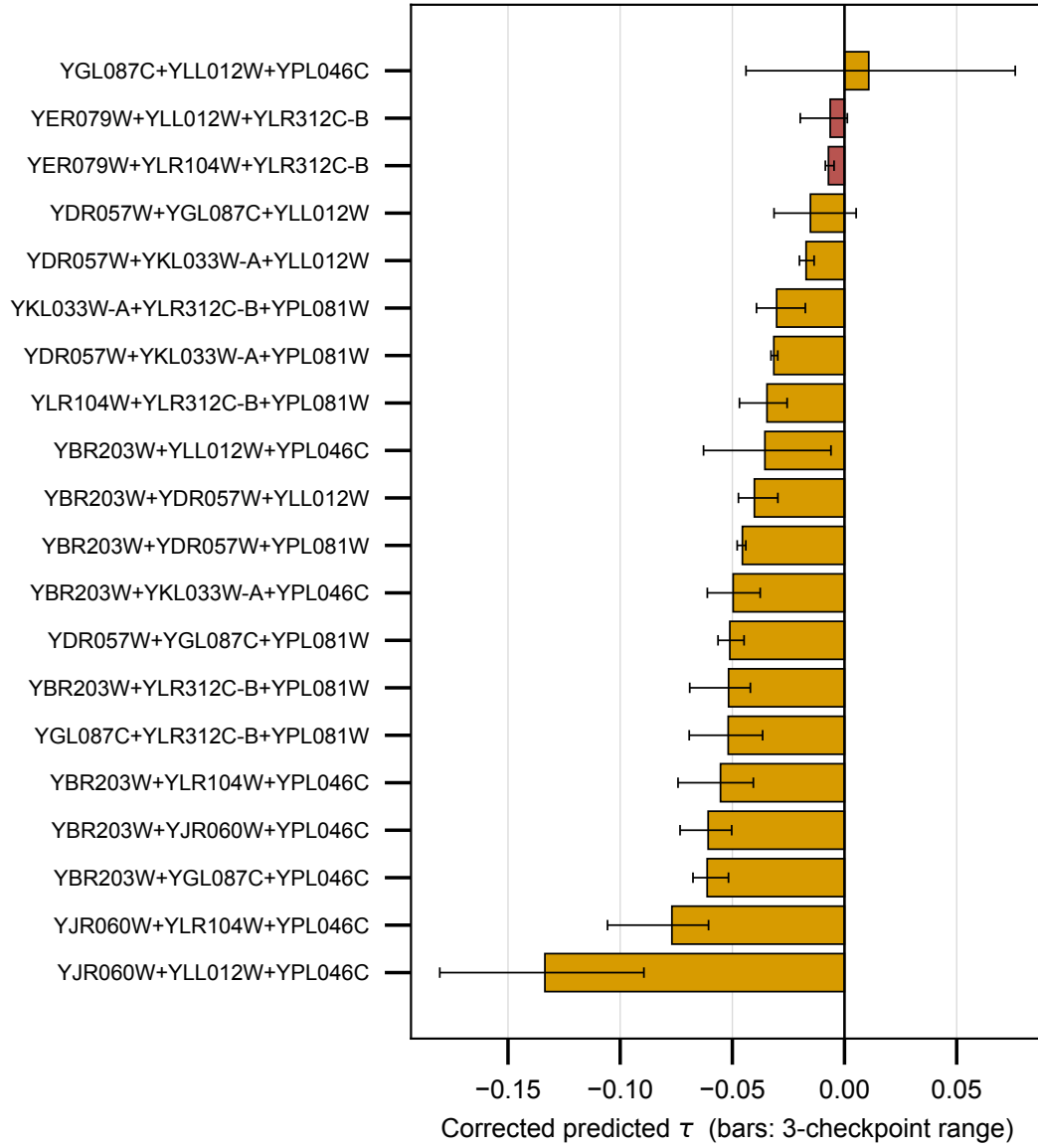


Figure 2: The 20 build-list triples re-ranked, bars spanning the three checkpoints. Red marks a triple containing a gene with zero trigenic training records, which on this list is YER079W alone.

4 What the model can still be used for $\times$

Every confident prediction in Section 3 is negative. That suggests using the model to nominate negative trigenic interactions rather than positive ones, and it is worth testing rather than assuming, because the label is already skewed negative: 59.4 percent of the 376,732 records have τ below zero and the mean is -0.0080 . A squared-error fit shrinks toward the mean, so a model that had learned nothing beyond it would also emit mostly negative predictions. That most predictions are negative is therefore not evidence of anything.

4.1 The measurement that is evidence $\times$

Retrieval, scored against the published call rather than a bare magnitude cut. Kuzmin 2018 defines a negative trigenic interaction as a conjunction, and the sidedness is deliberate: “we used an established interaction magnitude cut-off for digenic interactions ($p < 0.05, |\varepsilon| > 0.08$) and trigenic interactions ($p < 0.05, \tau < -0.08$)”, with positives excluded because “We focused exclusively on the analysis of deleterious negative trigenic interactions.” The symmetric $|\tau| > 0.08$ belongs to Kuzmin 2020 and the 2021 protocol, which do score positives.

The conjunct is not cosmetic. On this build 29,712 records clear $\tau < -0.08$ and only 5,675 also clear $p < 0.05$, so magnitude alone is about five times more permissive than the published call. The p-value comes from the LMDB rather than the label table, and it is the significance of the unadjusted triple-mutant ε computed at the digenic scoring stage, which makes it a usable filter and not a statistic about τ .

Model	P@100	enrichment	average precision
B1 additive ridge	0.240	14.1 $\times$	0.0731
B5 nonlinear MLP	0.290	17.0 $\times$	0.0920
CGT M01	0.320	18.7 $\times$	0.1258
CGT M02	0.380	22.3 $\times$	0.1362
CGT M03	0.340	19.9 $\times$	0.1304

Table 3: Retrieving published negative trigenic interactions, $\tau < -0.08$ and $p < 0.05$, on the random-over-records test split. The base rate is 1.707 percent, so 38 of the best checkpoint’s 100 most negative predictions are real calls.

The transformer’s advantage over the additive null is proportionally larger here than on correlation: average precision 0.136 against 0.073, where on Pearson the same comparison was 0.438 to 0.455 against 0.400. Retrieving the negative tail is what the extra capacity buys.

4.2 It does not survive the disjoint split $\times$

The same measurement on the query-pair-disjoint split, where only the baselines have been refit. The disjoint test base rate is 2.739 percent, and precision at 100 is 0.050 for the additive ridge and 0.040 for the nonlinear baseline, enrichments of 1.8 $\times$ and 1.5 $\times$ against 14.1 $\times$ and 17.0 $\times$ on the random split.

That is close to chance. The stringent tier behaves the same way, 29.3 $\times$ on the random split for the best checkpoint against 1.7 to 2.3 $\times$ for the baselines on the disjoint one.

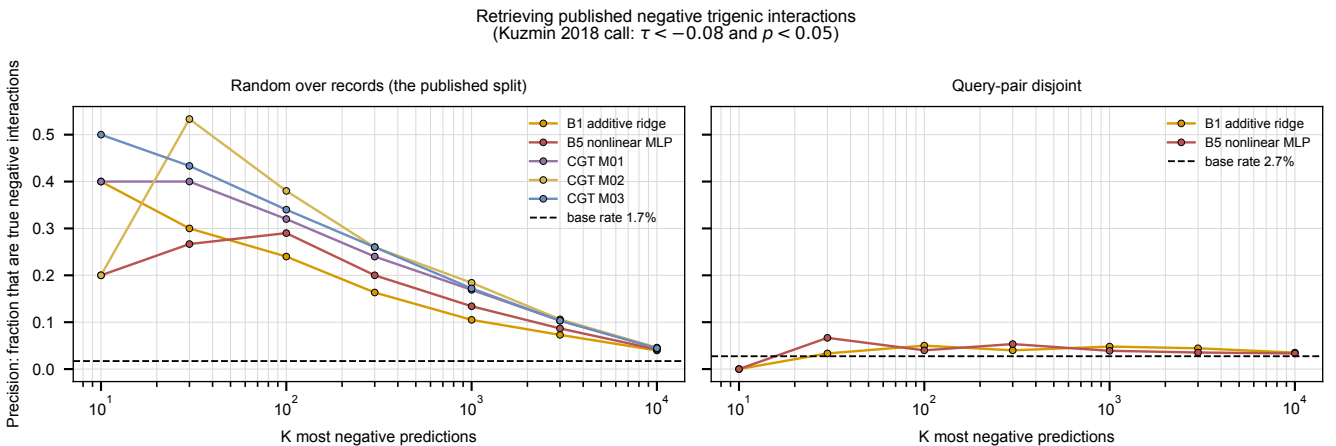


Figure 3: Precision at K for retrieving published negative trigenic interactions, on both splits, with the base rate marked. On the right the curves sit on the base rate.

4.3 What this supports ✗

Using the model to nominate negative interactions is defensible on the published split, and it fits what the model can do better than nominating positives. On the build list 17 of 20 triples have all three checkpoints agreeing and every one is negative, while the single positive is not agreed.

The published split is not where panel selection lives. On combinations whose query double was never screened, the baselines retain almost nothing and the transformer has not been refit. An earlier version of this section reported the disjoint result as retrieval surviving several fold weaker; that reading came from a magnitude-only threshold and was too generous.

5 Established, and not ✗

5.1 Established ✗

Two independent defects invalidated the design-space predictions, and both are confirmed by reproduction rather than argued from symptoms. Every stored prediction is exactly recoverable under one index map and not the other, so the model is deterministic and the runs were scoring different genes.

`inference_1` is unaffected by either defect. The panel-12 and panel-24 selections, which came from `inference_3`, are invalid.

The corrected panel predictions carry no confidently positive triple. The corrected values correlate with the ones the list was ordered by at 0.115 over the shared combinations, so the old ranking cannot be reused.

The model retrieves strong negatives well above base rate and better than an additive null, and by a proportionally wider margin than its correlation suggests.

5.2 Not established ✗

Whether the model ranks novel combinations well. Every number here comes from checkpoints trained on a split that is random over records. On that split an additive null reaches 0.400 and a model that ignores the third gene entirely reaches 0.390, and on a query-pair-disjoint split the additive null falls to 0.127 ± 0.033 . The transformer has not been refit on that split. These are the model's predictions computed correctly, which is a different claim from the model being right.

The retrieval enrichment under a disjoint split. Measured for the baselines only, where it falls several fold. Not measured for the transformer.

Whether the plate's duplicate is a duplicate. SPH1 and YLR312C-B name one locus, which is a fact about the annotation. Whether the two wells hold the same deletion or one is mislabeled is a question for the bench.

How far the gene-set filter costs us. Our gene set is built from features of type `gene`, so 53 of the 5,493 genes Costanzo measured, including two real loci on the original panel, cannot be represented at all. Whether that exclusion is right is a modeling decision that has not been revisited.

5.3 Next ✗

1. Pin the gene index space with the build and load it, rather than recomputing it from a live genome. The guard of Section 2.4 detects the failure; this removes it.
2. Intersect any candidate gene list with the model's gene space before generating combinations, and fail loudly on a name the index cannot resolve rather than dropping a perturbation.
3. Resolve gene names through the shared resolver wherever a count or a lookup is keyed on a systematic name, since the same alias miss has now produced both a wrong index and a wrong training-support audit.
4. Regenerate the panel selections from a correctly indexed run if a new selection is wanted, rather than reordering the existing one.
5. Retrain on the query-pair-disjoint split, which is the measurement every claim here is currently waiting on.